

Homework 2

Due: @11:55PM, Monday, October 7th

Name: _____ Shikha Nukala _____ Uniqname: _____ desigirl _____

1. Submit a pdf of your typed or handwritten homework on Gradescope.
2. Your answers should be neat, clearly marked, and concise. Typed work is recommended, but not required unless otherwise stated. Show all your work where requested, and state any special or non-obvious assumptions you make.
3. You may discuss your solution methods with other students, but the solutions you submit must be your own.
4. **Late Homework Policy:** Submissions turned in by 1:00 am the next day will be accepted but with a 5% penalty. Assignments turned in between 1:00 am and 11:55 pm will get a 30% penalty, and any submissions made after this time will not be accepted.
5. When submitting your answers to Gradescope you need to indicate what page(s) each problem is on to receive credit. The grader may choose not to grade the homework if answer locations are not indicated.
6. After each question (or in some cases question part), we've indicated which lecture number we expect to cover the relevant material. So "(L7)" indicates that we expect to cover the material in lecture 7 (but depending on your lecture, it may have been covered later)
7. **The last question is a group question.**
 - You may do it in a homework group of up to two students including yourself (yes, you can do them by yourself if you wish).
 - If you work in a group of two for these questions, list the name of the student you worked with in your assignment. Further, we suggest that you not split these problems up but rather work on the problem as a group.
 - Turn these group questions in as part of your individual submission.
 - **For these questions (and these questions ONLY) you are allowed to copy/paste solutions from the other student in your homework group.**
 - It is an honor code violation if a student is listed as contributing who did not actually participate in working on that problem.

Problem 1: Short answer questions (20 points)

1. An FSM has 3 input bits, 18 states, and 4 output bits . What total size ROM is needed to implement the FSM? **(8 points) (L10)**

3 input bits. 18 states to bits is $\log_2(18)$ which is equal to 5 bits per state. We get input combinations of $2^{(3+5)}$ which is 256 bits. Additionally we know there are 4 output bits per state and 5 bits per state, therefore we need 9 bits in memory for each memory location in the FSM. Therefore we need a total of $256 * 9 = 2304$ bits in memory to implement the FSM.

2. Global variables do appear in the symbol table of an object file **(2 points) (L7)**

Circle One: **True** or **False**

3. A D-latch can be made out of **(select all that apply) (4 points) (L9)**

- a. 3x NOR gates and 1x NOT gate
- b. 2x NOR gates, 1x AND gate, and 1x NOT gate
- c. 3x NOR gates, 2x NOT gate
- d. 2x NOR gates, 2x AND gate, and 1x NOT gate**
- e. None of the above

4. A D-Latch only updates its value on the rising edge of the clock signal. **(2 points) (L9)**

Circle One: **True** or **False**

5. Caller save is always better than callee save because in caller save, leaf functions do not need to save registers even if they use them (Leaf functions do not call other functions) **(2 points) (L6)**

Circle One: **True** or **False**

6. When building a single cycle processor, the clock period must be at least as long as the execution latency of the longest (slowest) instruction. **(2 points) (L10)**

Circle one: **True** or **False**

Problem 2: I've had my fill of LC2K (10 points)

Using only `.fill` commands, write an LC2K program which loads the value 5 into reg 1, loads the value 7 into reg 2 and computes bitwise NOR of those two values, leaving the result in register 5 in as few lines as possible (you need to actually have the computer NOR those two values together!) (L5)

```
.fill 0      # register 0 remains 0
.fill 5      # loads 5 to first available location
.fill 7      # loads 7 to next available location
.fill 0      # filler for third location
.fill 0      # filler for fourth location
.fill -8     # nor of 5 and 7 is -8
```

0101 = 5

0111 = 7

Nor = 1000 which is -8

Problem 3: Link Like You've Never Linked Before! (12 Points)

(L11)

Project 2 spec deals with linking in LC2K. Below is an LC2K assembly program and a partially complete object file. **Note: the machine code should be completed in hexadecimal, not decimal.** You are to fill in all the blanks of the object file (don't forget the first line). It is possible that one or more lines should remain blank.

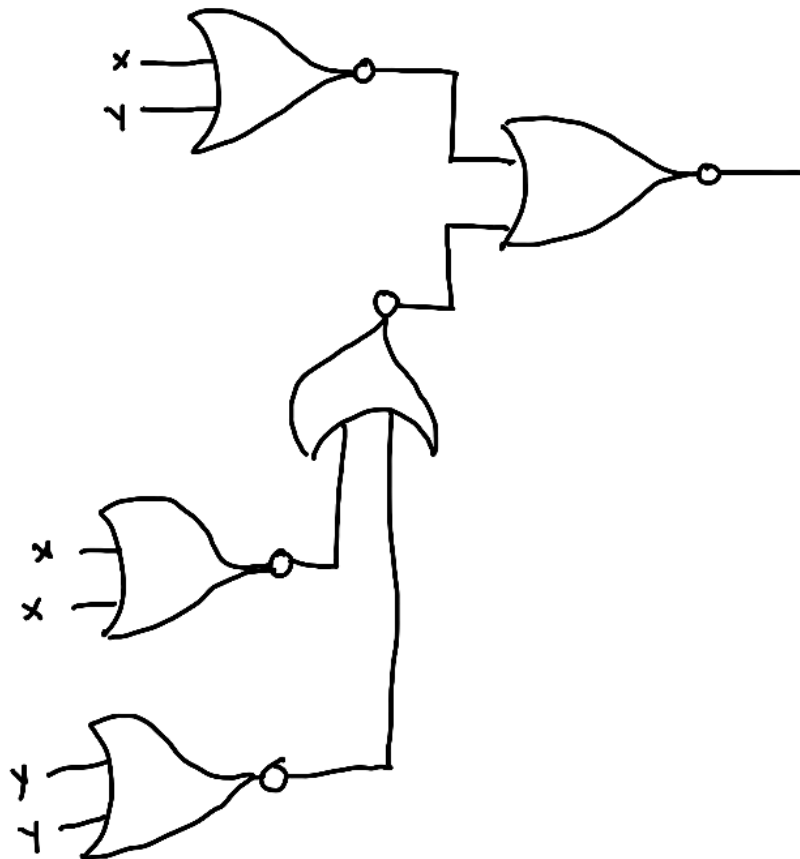
Main.as	Main.obj
	9 4 5 5
start lw 0 2 Five	0x820009
lw 0 1 Glob1	0x810000
beq 0 1 GlobB	0x1010003

next add 2 2 2	0x120002
beq 0 0 start	0x100FFFB
sw 0 1 GlobD	0xC10000
GlobB nor 1 2 1	0x4A0001
jalr 4 7	0x1670000
halt	0x1800000
Five .fill 5	5
One .fill Glob1	0
.fill 1	1
.fill next	3
	GlobB T 6
	Five D 0
	GlobD U 0
	Glob1 U 0
	One D 1
	0 lw Five
	1 lw Glob1
	5 sw GlobD
	1 .fill Glob1
	3 .fill next

Problem 4: Just a Bunch of Gates (14 points) (L9)

- Design a circuit that takes in two 1-bit inputs. If the two inputs are different then it outputs 1 otherwise it outputs a 0. You may only use only NOR gates. (You may express your answer either as a circuit diagram or using a Boolean expression) (4 points)

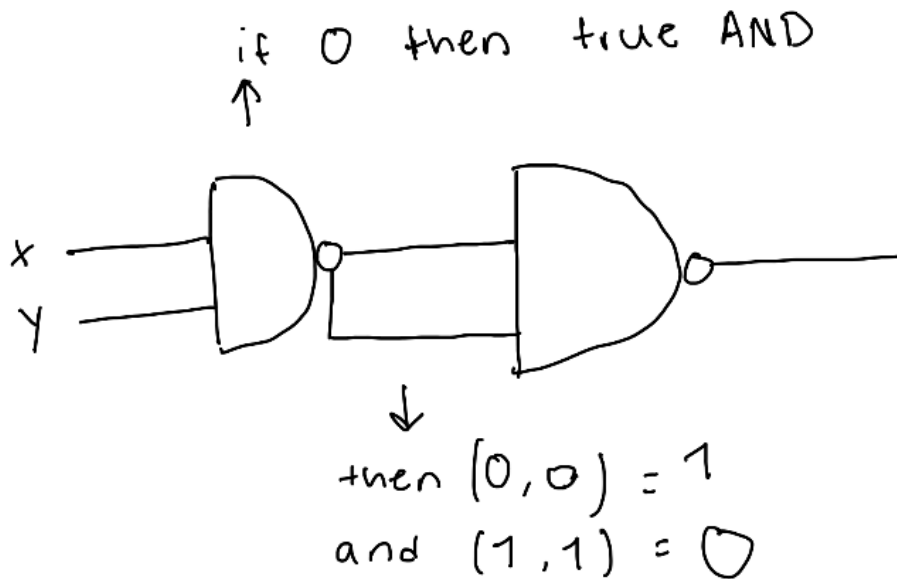
x	y	$\neg x$	$\neg y$	$x \text{ NOR } y$	$\neg x \text{ NOR } \neg y$	$(x \text{ NOR } y) \text{ NOR } (\neg x \text{ NOR } \neg y)$
0	0	1	1	1	0	0
0	1	1	0	0	0	1
1	0	0	1	0	0	1
1	1	0	0	0	1	0



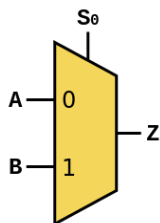
1

- Design an AND gate out of 2-input NAND gates. (2 points)

NAND gate 1 if (0,0) (0,1) (1,0)
0 if (1,1)



3. Fill in the truth table below for the 2-to-1 Mux below (4 points)



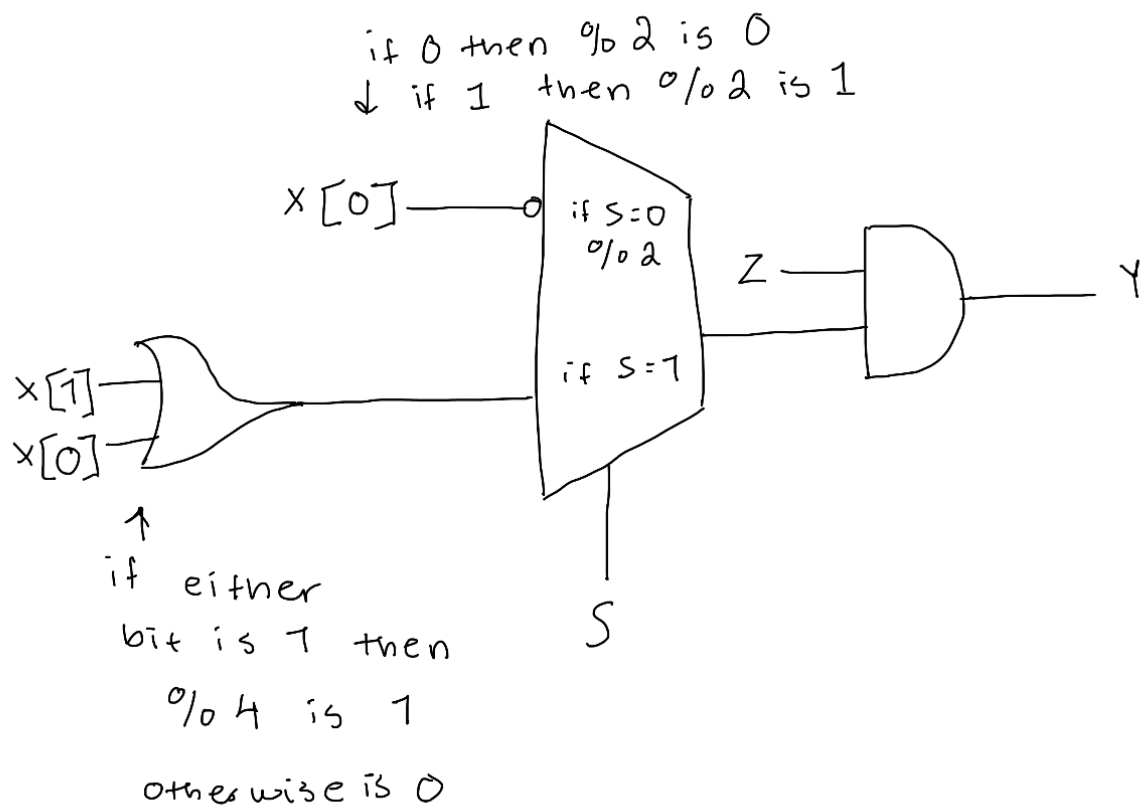
S	A	B	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1

1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

4. Given the following code, design a circuit which has 34 bits of input ($X[31:0]$) and the select bit S and the single bit Z , and has 1 bit of output (Y). For this problem you should consider the input X to be an unsigned number.

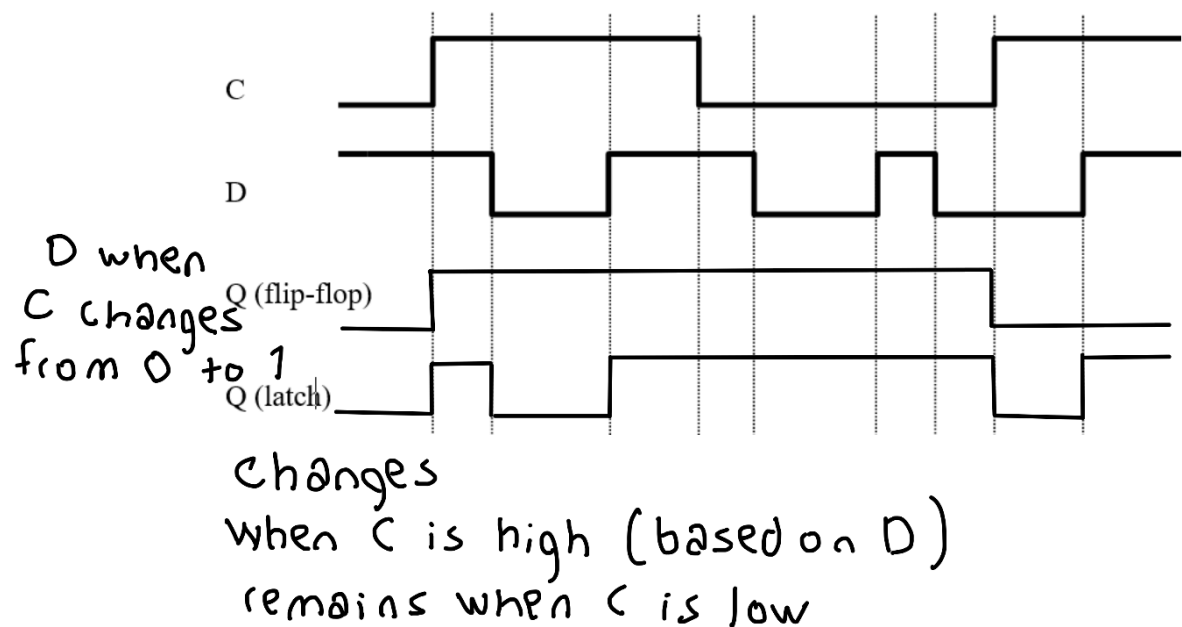
You can use a single bit 2x1 mux, one AND gate and one OR gate. (4 points)

```
bool HW3Func(uint_t32 x, bool S, bool Z){
    bool y;
    if(S){
        y = (x % 4 != 0);
    }
    else{
        y = (x % 2 != 0);
    }
    return y & Z;
}
```



Problem 5: Latches and flip-flops (8 points)

Complete the timing diagrams for a D latch and D flip-flop given the provided inputs. The flip-flop is a positive-edge triggered. For the latch we are using "C" for the "G" input (which is fairly common). Assume both the latch and the flip-flop have an initial value of zero. (L9)



Problem 6: Finite State Machine (16 Points) (L10)

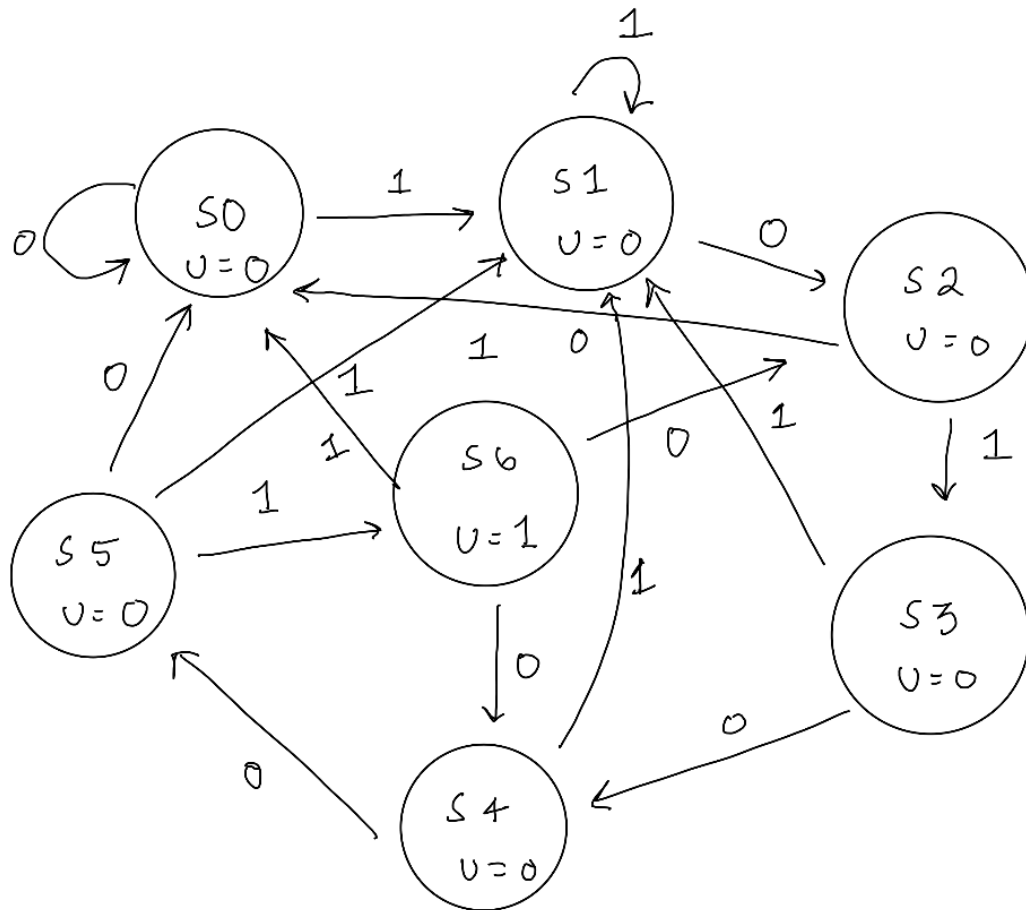
Design a Moore machine where the output Y goes high (=1) when the last six bits of the input X were 101001:

6th to last bit seen = 1
5th to last bit seen = 0
4th to last bit seen = 1
3rd to last bit seen = 0
2nd to last bit seen = 0
Last bit seen = 1

Your machine must be designed to use the least number of states possible. Your reset state should be a starting state in which no bits of X have been read. Your FSM should then proceed to read an infinite number of bits producing an output of 1 whenever the 6 most recent bits match the pattern 101001 and otherwise to produce an output of 0.

Other doc

1. Use this page to draw your FSM (6 points)



2. Complete the following state transition table based on your FSM. (10 points)

<u>State</u>	<u>Next State</u>		<u>Output</u>
	(Input X = 0)	(Input X = 1)	
S0	S0	S1	0
S1	S2	S1	0
S2	S0	S3	0
S3	S4	S1	0
S4	S5	S1	0

S5	S1	S6	0
S6	S2	S0	1

Problem 7: SC Datapath Performance (20 Points) [Group] (L10)

Consider the single-cycle datapath from lecture with the following delays:

Read memory	10 ns
Write memory	16.5 ns
Read register file	4 ns
Write register file	7 ns
ALU	5 ns
All other operations	0 ns

1. What is the clock period for the single-cycle processor if we only had to support add, nor, beq and sw?

Nor = read instruction, read register, write register, ALU, other operations - no memory access

$$10+4+7+5 = 26$$

Beq = read instruction, read register, 2x ALU, other operations = $10+4+10 = 24$

Sw = read instruction, read register, ALU, write memory, other operations = $10+4+5+16.5 = 35.5$

Add = read instruction, read register, write register, ALU, other operations = $10+4+7+5 = 26$

Clock period is the longest instruction which is sw which takes **35.5 ns**.

2. What is the clock period for the single-cycle processor if we support all LC2K instructions except jalr? [5]

Nor = read register, write register, ALU, other operations - no memory access

$$10+4+7+5 = 26$$

Beq = read register, 2x ALU, other operations = $10+4+10 = 24$

Sw = read register, ALU, write memory, other operations = $10+4+5+16.5 = 35.5$

Add = read register, write register, ALU, other operations = $10+4+7+5 = 26$

Lw = read register, writer register, ALU, read memory, other operations = $10+4+7+10+5 = 36$

Noop = 0

Halt = 0

Lw is the longest instruction taking **36 ns** so this is the clock period.

3. If we can decrease the delay for one of the operations by 10%, which operation should we apply it to in order to improve performance the most? What is the clock period after improvement? [5]

Decrease delay for read memory which becomes 9. Since lw is the longest-time instruction we can decrease this to 35 ns for a since lw instruction execution. Then the clock period would be determined by the next longest instruction which is sw, so the clock period would be capped as **34.5 ns**.

Consider a benchmark of 1000 instructions. The instructions consist of:

instruction	percentage
add / nor	35%
sw	20%
lw	25%
beq	15%
noop / halt	5%

With the original latency, what is the execution time of this benchmark on the single-cycle processor? [5]

$36 \times 1000 = 36000 \text{ ns}$.